Exercice 5 (Labyrinthe – Python)

Vous êtes dans un labyrinthe. Vous devez retrouver *le plus court chemin* du point de départ `S` jusqu'à la sortie `E`. Pour ce faire, on veut déterminer la séquence de déplacements à faire pour rejoindre la sortie. Le labyrinthe est représenté comme une grille de caractères, où chaque case de la grille peut être :

- `#` : un mur (non franchissable),
- `.` : une case libre donc franchissable,
- `S` : le point de départ (start),
- `E` : la sortie (end).

On peut se déplacer **uniquement dans les 4 directions orthogonales** : `U` = haut (up), `D` = bas (down), `L` = gauche (left), `R` = droite (right). Voici un exemple de labyrinthe suivant cette représentation :

```
exemple_labyrinthe = [
    "######",
    "#.S..#",
    "#.##.#",
    "#....#",
    "###E##"
]
```

Une façon de s'échapper de ce labyrinthe en faisant un minimum de déplacements serait de faire les déplacements `"LDDRRD"`. Notez que pour certains labyrinthes il est possible d'avoir plusieurs chemins ayant une longueur minimale : retourner l'un d'entre eux est suffisant (dans l'exemple, `"RRDDLD"` marche également).

Question 1 Pourquoi l'utilisation de l'algorithme BFS est particulièrement adaptée pour ce problème du plus court chemin ? Expliquer.

Question 2 Écrire une fonction Python `trouver_debut_sortie(labyrinthe)` prenant en paramètre un labyrinthe possédant la structure décrite plus haut, et retournant deux tuples Python correspondant à la position (les coordonnées) du départ et de la sortie dans le labyrinthe. *Rappels/Indice : Pour retourner deux éléments a et b, on écrit* `return a, b`*. On rappelle aussi qu'en Python,* ***une chaîne de caractères peut être vue comme une liste de caractères****. Ainsi, le labyrinthe n'est rien d'autre qu'une liste de listes...*

Question 3 Compléter le code Python de la fonction `plus_court_chemin(labyrinthe)`, partout où des champs « `____` » apparaissent. Quelques indications :

- **Vous devez répondre dans votre copie et non sur le sujet** : vous pouvez pour ce faire écrire le numéro de la ligne suivi de la ligne complétée.
- Le nombre de « `_` » est choisi aléatoirement et ne reflète donc pas le nombre de caractères attendu.
- Des précisions sont données sous la forme de commentaires Python pour aider à la compréhension.

```python
from collections import deque

def plus_court_chemin(labyrinthe):
    n, m = len(labyrinthe), len(labyrinthe[0])
    debut, sortie = trouver_debut_sortie(labyrinthe)

    deja_visitees = [[False]*m for _ in range(n)] # Structure de donnees pour garder trace des cases deja visitees
    deplacements = {}  # Dictionnaire gardant trace des differents deplacements realises : (x, y) -> (px, py, direction)

    # On initialise la file et on visite la case de depart
    file_ = deque() # file est un mot reserve en Python, on utilise file_
    file_.append(debut)
    deja_visitees[debut[0]][debut[1]] = True

    # Les deplacements possibles sont les directions orthogonales
    directions = [(-1,0,'U'), (1,0,'D'), (0,-1,'L'), (0,1,'R')]

    while file_:
        x, y = file_.popleft()
        if (x, y) == sortie:
            break
        for dx, dy, lettre in directions:
            nx, ny = x + dx, y + dy
            if _________________________: # Condition(s) pour ne pas chercher en dehors du tableau
                if _________________________: # Condition(s) pour pouvoir visiter la case (nx, ny)
                    deja_visitees[___][___] = _____
                    file_.append(_____)
                    deplacements[(nx, ny)] = (x, y, lettre)

    # Reconstruction du chemin en partant de la sortie et en remontant jusqu'au debut
    chemin = []
    case_courante = sortie
    while case_courante != debut:
        px, py, lettre = ____________
        chemin.append(__________)
        case_courante = (px, py)

    return ''.join(reversed(chemin)) # On retourne les deplacements dans le bon ordre
```

Question 4 Quelle est la complexité temporelle de cet algorithme ? Justifier.